

Technical Report

Design and Verification of an 8-bit Dadda Multiplier

Reece Dobro

May 25, 2026

Abstract

This development work consists of developing and testing an 8-bit multiplier in the Verilog programming language using AMD Vivado. Testing consists of 10 sets of 8-bit values chosen to thoroughly exercise the functionality of the Dadda multiplier across a range of input conditions, including edge cases. The results of the test values, extracted through a testbench, confirmed that the multiplier produces correct 16-bit outputs for all tested input combinations. The implementation and theoretical background of the 8-bit Dadda multiplier are thoroughly detailed and validated through simulation.

1 Introduction, Background, and Theory

The goal of this project is to build, program, and test an 8-bit binary multiplier in Verilog using the Dadda Multiplier design. The program, *Dadda8x8.v*, is made up of three modules and a testbench that was run through AMD Vivado. The first module is a single bit full adder, the second module is a single bit half adder, and the third is a Dadda reduction tree combined with a 16-bit ripple carry adder. The testbench allows for the multiplier to be tested and deemed correct using 10 chosen sets of test values. This project demonstrates the implementation of complex multiplication in Verilog through logic reduction algorithms and the methodology for verifying designs using a testbench and certain input cases.

The first module, *FA*, is shown in Figure 1. The module is a single bit full adder at the gate level, which is composed of two XOR gates, two AND gates and a single OR gate. The second module, *HA*, is shown in Figure 2. It is a single bit half adder at the gate level, which is composed of a single XOR gate and an AND gate.

```
module FA(  
    input A,  
    input B,  
    input Cin,  
    output S,  
    output Cout  
);  
    wire net1; //output from xor of (a,b)  
    wire net2; //output from and (a,b)  
    wire net3; //output from cin and net1  
  
    xor u1(net1,A,B);  
    xor u2(S,net1,Cin);  
    and u3(net2,A,B);  
    and u4(net3,Cin,net1);  
    or u5(Cout,net3,net2);  
endmodule
```

Figure 1: Source code of the Full Adder module

```

module HA( //Half Adder
    input A,
    input B,
    output S, //Sum bit
    output Cout //Carry out bit
);

    xor u1(S,A,B);
    and u2(Cout,A,B);
endmodule

```

Figure 2: Source code of the Half Adder module

1.1 Dadda Multiplier Development

The process of multiplying two binary numbers using the Dadda design has 3 main processes, all of which are featured in the 3rd and final module, *Dadda8x8*. The first part is creating partial products. When multiplying two numbers, partial products are used to break up the large products of numbers into smaller groups. For a Dadda multiplier, these groups are 1 bit, with each bit of the multiplier against each bit of the multiplicand. As there are two 8-bit vectors for the inputs, this results in a total of 64 partial products. In total, **64 AND gates** were used for this project. For each partial product, an AND gate is required, so 64 AND gates are required. For organizational and readability purposes, an 8 by 8 array is used to hold all of the partial products. While each partial product could be programmed manually, line by line, it would be time-intensive and prone to errors. Due to this, a generate function in Verilog is used to program all 64 AND gates with their proper inputs and outputs. This is shown in Figure 3.

```

wire AB[7:0][7:0]; // AB[i][j] = A0B0, A0B1, etc.
genvar i, j;
generate
    for (i = 0; i < 8; i = i + 1)
    begin
        for (j = 0; j < 8; j = j + 1)
        begin
            and (AB[i][j], a[i], b[j]);
        end
    end
endgenerate

```

Figure 3: Verilog generate function for 64 AND gates

The next step in multiplying two binary numbers using the Dadda design is to reduce the partial product summations to at most two rows. Reducing the partial product summation, for the Dadda method, involves placing full adders and half adders in columns when the column height exceeds a certain threshold. The number of columns are equal to the size (in binary) of the multiplicand (in this case, *A*) added with the size of the multiplier, (in this case, *B*). This is because partial products are shifted left based on the bit positions of the multiplicand and multiplier. Once the last column is completed, the process is repeated from the first column using a new threshold until all columns have a height of at most two. The reduction process for multiplying two 8-bit vectors together is shown in Figure 4.

As shown in the figure, the only complex functions needed to create a Dadda reduction tree are a full adder and half adder. In total, 4 levels of reduction are required before getting to an output whose final sum can be found with a ripple carry adder. The last grid shows the final two rows of sums and carries before the final addition.

The final addition is the last part of multiplying two binary numbers. Due to its simplicity and the fact that a half adder and a full adder module already exist in the program due to the previous step of Dadda reduction, a ripple carry adder was chosen as the method to compute the final sum. The implementation is shown in Figure 5. The first bit of the sum, $prod[0]$ is not included as it is equivalent to the AND of the first bit of a and the first bit of b . The ripple carry adder used 13 Full Adders and 7 Half Adders, while the Dadda partial product reduction used 35 Full Adders and 7 Half Adders. In total, this project used **48 Full Adders**, **8 Half Adders**, and **64 AND gates**.

1.2 The Testbench

The implementation in Verilog of the testbench is shown in Figure 6. It follows a simple structure, displaying the product of 10 sets of test values. The reasoning for these test vectors is in the following section. The display statements allow for the values to be easily recorded and compared to the correct values. 100 nanoseconds were chosen as the period for each set of test values of the testbench as it gives each set of test values an equal amount of time to display. This improves clarity and presentation of the waveforms.

```

module testbench;
    reg [7:0] a, b;           // inputs to Dadda8x8
    wire [15:0] run;          // output of Dadda8x8
    Dadda8x8 ul(a,b,run);

    initial begin
        a = 8'b0000_0000;
        b = 8'b0000_0000;
        #100 $display("in a: %b, in b: %b, out: %b", a, b, run);
        a = 8'b0000_0000;
        b = 8'b1111_1111;
        #100 $display("in a: %b, in b: %b, out: %b", a, b, run);
        a = 8'b1111_1111;
        b = 8'b1111_1111;
        #100 $display("in a: %b, in b: %b, out: %b", a, b, run);
        a = 8'b0000_0001;
        b = 8'b1111_1111;
        #100 $display("in a: %b, in b: %b, out: %b", a, b, run);
        a = 8'b1111_1111;
        b = 8'b0000_0001;
        #100 $display("in a: %b, in b: %b, out: %b", a, b, run);
        a = 8'b1000_0000;
        b = 8'b0000_0001;
        #100 $display("in a: %b, in b: %b, out: %b", a, b, run);
        a = 8'b0101_0101;
        b = 8'b1010_1010;
        #100 $display("in a: %b, in b: %b, out: %b", a, b, run);
        a = 8'b0111_1111;
        b = 8'b0111_1111;
        #100 $display("in a: %b, in b: %b, out: %b", a, b, run);
        a = 8'b1111_0000;
        b = 8'b1111_0000;
        #100 $display("in a: %b, in b: %b, out: %b", a, b, run);
        a = 8'b0000_1111;
        b = 8'b0000_1111;
        #100 $display("in a: %b, in b: %b, out: %b", a, b, run);
    end
endmodule

```

Figure 5: Testbench implementation in Verilog

1.3 Test Values and Justification

In order to test the 8×8 Dadda Multiplier, 10 sets of test values were chosen. These values (in binary) and their justification are shown in Table 1. Edge cases, properties of multiplication, and typical operation scenarios were included to ensure the multiplier functions correctly under all conditions. It is important to note that the test value $a = 8'b\ 1111\ 1111$ and $b = 8'b\ 0000\ 0001$ can be used to verify

the commutative property, as their result can be compared with the test values $a = 8'b\ 0000\ 0001$ and $b = 8'b\ 1111\ 1111$.

a (8-bit vector)	b (8-bit vector)	Justification
0000 0000	0000 0000	Verifies zero-input multiplication
0000 0000	1111 1111	Verifies the multiplicative property of zero, $a \times 0 = 0$
1111 1111	1111 1111	Maximum product edge case
0000 0001	1111 1111	Verifies the identity property, $a \times 1 = a$
1111 1111	0000 0001	Verifies the commutative property, $a \times b = b \times a$
1000 0000	0000 0001	MSB $\times$ LSB, tests extreme bit multiplication
0101 0101	1010 1010	Alternating bit pattern to test carry propagation
0111 1111	0111 1111	Large numbers below max, tests typical operation
1111 0000	1111 0000	Tests higher 4x4 multiplication region
0000 1111	0000 1111	Tests lower 4x4 multiplication region

Table 1: 8-Bit Dadda Test Values Justification

2 Results & Analysis

The output of the Dadda Multiplier, (the 16-bit vector *sum*) is shown through Figures 6 and 7. Figure 6 shows the waveform output from AMD Vivado of the testbench and Figure 7 shows the output of the testbench through the TCL Console. These figures provide two different views of the outputs. Figure 6, the output of the waveforms, demonstrates how the changes in the test values cause a change in the *sum* values. Additionally, Figure 6 shows the similarities in the sets of test values chosen. Figure 7 allows for the raw outputs and test values to be analyzed and compared to the expected outputs of the program. The data generated from Figure 7 was compiled into Table 2, along with the values from Table 1, for simplicity and comparison. For readability purposes, the outputs are expressed in hexadecimal.

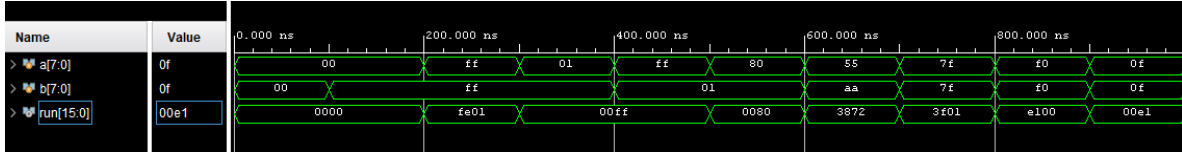


Figure 6: Waveforms Output of the Testbench

```

in a: 00000000, in b: 00000000, out: 0000000000000000
in a: 00000000, in b: 11111111, out: 0000000000000000
in a: 11111111, in b: 11111111, out: 1111111000000001
in a: 00000001, in b: 11111111, out: 0000000011111111
in a: 11111111, in b: 00000001, out: 0000000011111111
in a: 10000000, in b: 00000001, out: 0000000010000000
in a: 01010101, in b: 10101010, out: 0011100001110010
in a: 01111111, in b: 01111111, out: 0011111100000001
in a: 11110000, in b: 11110000, out: 1110000100000000
in a: 00001111, in b: 00001111, out: 0000000011100001

```

Figure 7: TCL Console Output of the Testbench

8-bit Inputs		16-bit Product	
a	b	Program Output	Expected Output
0000 0000	0000 0000	0x0000	0x0000
0000 0000	1111 1111	0x0000	0x0000
1111 1111	1111 1111	0xFE01	0xFE01
0000 0001	1111 1111	0x00FF	0x00FF
1111 1111	0000 0001	0x00FF	0x00FF
1000 0000	0000 0001	0x0080	0x0080
0101 0101	1010 1010	0x3872	0x3872
0111 1111	0111 1111	0x3F01	0x3F01
1111 0000	1111 0000	0xE100	0xE100
0000 1111	0000 1111	0x00E1	0x00E1

Table 2: Dadda Multiplier Verification

As shown by Table 2, the output of the Dadda multiplier matches the expected output. This successful verification demonstrates that the multiplier is properly implemented and functions as intended. By passing the given test values, the multiplier is shown to correctly handle edge cases, carry propagation, maximum input conditions, typical operation, and properties of multiplication.

A potential weak point in the data is that only 10 test cases are used. Since there are 2^8 possible values for each of the 8-bit vectors a and b , there are 2^{16} possible test values, with the 10 sets chosen covering only around 0.015% of the total possible test values. However, it would be impractical to use all possible test values to test the completed module due to the length of time to manually verify and document each output. Potential sources of error include incorrect wiring in the partial product reduction stages or misalignment of sums and carries from full adders in the columns closer to the MSB. While no errors were observed in the selected test cases after debugging was complete, exhaustive testing would be required to guarantee correctness for all possible 8-bit input combinations. The only bug that did occur was incorrectly entering *FAS*[19] instead of *FAS*[10] into a Full Adder instance *FA20*. After analyzing the incorrect test case outputs, the error was quickly resolved, demonstrating the reliability of the chosen test cases.

As the outputs from the program matched the expected results, the code and the logical structure for the 8×8 Dadda Multiplier are deemed successful and reliable.

3 Conclusions

This project demonstrated how logical functions and gate-level designs can be implemented and tested in Verilog. It also presented an implementation of an 8-bit Dadda Multiplier constructed from only full adders, half adders, and AND gates. However, while the program was successful, there are areas that could be improved for future design iterations. The test cases focused solely on functional correctness. Time and efficiency were not taken into account, and separate analyses should be conducted to determine what changes could be made to the design to optimize performance. A possible future study could take into account timing path delays and efficiency metrics when designing test cases. It is also possible that an FPGA could be used to gather real-time delay statistics. With that being said, this project was able to demonstrate the capabilities of Verilog to test complex logic for a large multiplier, while also creating a module that will be useful for future architecture integrations and one that works well with existing components. Instead of needing to perform development at the physical prototyping level, which would require a large amount of resources such as discrete logic gates, wires, and chips, this project shows how a design can be fully tested and implemented digitally with minimal use of physical resources.